

Tổng Quan Chi Tiết về Jetson, Raspberry Pi và Các Định Dạng Model (PT, ONNX, TensorRT Engine)

Ngày 26 tháng 4 năm 2026

Mục lục

1	Giới thiệu	3
2	Nền tảng phần cứng	4
2.1	NVIDIA Jetson Platform	4
2.1.1	Tổng quan	4
2.1.2	Các dòng Jetson phổ biến	4
2.1.3	Kiến trúc phần cứng Jetson	5
2.1.4	Hệ sinh thái phần mềm của Jetson	6
2.1.5	Ưu điểm của Jetson	6
2.1.6	Nhược điểm của Jetson	6
2.2	Raspberry Pi	7
2.2.1	Tổng quan	7
2.2.2	Các dòng Raspberry Pi phổ biến	7
2.2.3	Đặc điểm kỹ thuật	8
2.2.4	Các phương án mở rộng AI cho Raspberry Pi	8
2.2.5	Hệ điều hành và phần mềm	9
2.2.6	Ưu điểm của Raspberry Pi	9
2.2.7	Nhược điểm của Raspberry Pi	10
2.3	So sánh Jetson và Raspberry Pi	10
3	Pipeline triển khai AI	11
3.1	Tổng quan quy trình	11
3.2	Chi tiết từng bước	11
3.2.1	Chuẩn bị dữ liệu	11
3.2.2	Huấn luyện trên môi trường phát triển	12
3.2.3	Export sang ONNX	12
3.2.4	Tối ưu hoá với TensorRT	13
3.2.5	Inference trên thiết bị	13
4	Các định dạng mô hình	15
4.1	PyTorch Model (.pt / .pth)	15
4.1.1	Khái niệm và cấu trúc	15
4.1.2	Cấu trúc nội bộ của file .pt	16
4.1.3	Sử dụng model .pt với YOLOv8	16
4.1.4	Ưu điểm của .pt	17
4.1.5	Nhược điểm của .pt	17
4.2	ONNX Model (.onnx)	17
4.2.1	Khái niệm và lịch sử	17

4.2.2	Cấu trúc của file ONNX	17
4.2.3	Kiểm tra và visualize ONNX model	18
4.2.4	ONNX Simplifier	18
4.2.5	ONNX Opset và tương thích	19
4.2.6	Các ONNX Execution Providers	19
4.2.7	Ưu điểm của ONNX	19
4.2.8	Nhược điểm của ONNX	20
4.3	TensorRT Engine (.engine / .trt)	20
4.3.1	Khái niệm và vai trò	20
4.3.2	Quy trình tối ưu hoá của TensorRT	20
4.3.3	Dynamic Shapes trong TensorRT	22
4.3.4	Inference với TensorRT C++ API	22
4.3.5	Kiểm tra hiệu năng với trtexec	23
4.3.6	Tương thích phiên bản	23
4.3.7	Ưu điểm của TensorRT Engine	23
4.3.8	Nhược điểm của TensorRT Engine	24
5	So sánh các định dạng mô hình	25
5.1	So sánh tổng quan	25
5.2	So sánh hiệu năng thực tế	25
5.3	Khi nào nên dùng định dạng nào	25
5.3.1	Dùng .pt khi	25
5.3.2	Dùng .onnx khi	26
5.3.3	Dùng .engine khi	26
6	Ứng dụng thực tế	27
6.1	Ứng dụng của Jetson	27
6.1.1	Xe tự hành và Robot	27
6.1.2	Hệ thống Camera AI thông minh	27
6.1.3	Smart City và Giám sát giao thông	27
6.1.4	Y tế và Thiết bị y tế	28
6.1.5	Agriculture AI	28
6.2	Ứng dụng của Raspberry Pi	28
6.2.1	IoT và Home Automation	28
6.2.2	Edge AI nhẹ với Accelerator	28
6.2.3	Giáo dục và Nghiên cứu	28
6.2.4	Media và Entertainment	28
7	Các lưu ý và thực hành tốt nhất	29
7.1	Lưu ý khi chọn phần cứng	29
7.2	Lưu ý khi chuyển đổi định dạng model	29
7.3	Tối ưu hoá thêm sau TensorRT	29
8	Kết luận	31

Chương 1

Giới thiệu

Trong bối cảnh phát triển mạnh mẽ của Trí tuệ nhân tạo (AI) và thị giác máy tính (Computer Vision), nhu cầu triển khai các mô hình học sâu (Deep Learning) trực tiếp tại thiết bị đầu cuối (Edge) ngày càng trở nên cấp thiết. Thay vì gửi toàn bộ dữ liệu lên cloud để xử lý, Edge AI cho phép thiết bị thực hiện suy luận (inference) ngay tại chỗ, mang lại các ưu điểm về độ trễ thấp, bảo mật dữ liệu tốt hơn và khả năng hoạt động độc lập với kết nối mạng.

Hai nền tảng phần cứng phổ biến nhất trong lĩnh vực này là **NVIDIA Jetson** và **Raspberry Pi**. Mỗi nền tảng phục vụ một nhóm ứng dụng khác nhau, với sự đánh đổi khác nhau giữa hiệu năng, chi phí và mức độ tiêu thụ điện năng.

Song song với phần cứng, việc lựa chọn định dạng mô hình phù hợp cũng ảnh hưởng lớn đến hiệu năng hệ thống. Ba định dạng phổ biến nhất hiện nay là:

- **.pt (PyTorch)**: Định dạng gốc dùng trong quá trình huấn luyện và nghiên cứu.
- **.onnx (Open Neural Network Exchange)**: Định dạng trung gian, hỗ trợ chuyển đổi giữa các framework.
- **TensorRT Engine (.engine)**: Định dạng tối ưu hoá cho inference trên GPU NVIDIA.

Tài liệu này trình bày chi tiết về kiến trúc, đặc điểm, ưu nhược điểm của từng nền tảng phần cứng; giải thích cơ chế hoạt động của từng định dạng mô hình; và hướng dẫn pipeline triển khai hoàn chỉnh từ quá trình huấn luyện đến khi chạy thực tế trên thiết bị.

Chương 2

Nền tảng phần cứng

2.1 NVIDIA Jetson Platform

2.1.1 Tổng quan

Jetson là dòng thiết bị nhúng (embedded computing) của NVIDIA, được thiết kế đặc biệt cho các ứng dụng AI và Deep Learning tại Edge. Không giống các vi xử lý nhúng thông thường, Jetson tích hợp cả CPU, GPU, và các bộ tăng tốc phần cứng chuyên dụng (như Tensor Cores và DLA — Deep Learning Accelerator) trên cùng một module SoM (System on Module). Điều này cho phép Jetson thực hiện các tác vụ suy luận AI với hiệu năng cao trong khi vẫn duy trì mức tiêu thụ điện tương đối thấp so với các máy chủ GPU đầy đủ.

2.1.2 Các dòng Jetson phổ biến

Jetson Nano

Jetson Nano là sản phẩm entry-level trong dòng Jetson, nhắm đến đối tượng học sinh, sinh viên và các nhà phát triển bắt đầu tìm hiểu AI tại Edge. Module này có giá thành phải chăng và có hai phiên bản: 2GB và 4GB RAM.

- **CPU:** Quad-core ARM Cortex-A57 @ 1.43 GHz
- **GPU:** 128-core NVIDIA Maxwell
- **RAM:** 4GB LPDDR4 (phiên bản đầy đủ)
- **TDP:** 5W / 10W
- **Khả năng AI:** ~472 GFLOPS (FP16)

Jetson Nano phù hợp với các tác vụ inference nhẹ như phân loại ảnh (image classification), phát hiện đối tượng (object detection) với model nhỏ, hoặc xử lý video độ phân giải thấp.

Jetson Xavier NX

Jetson Xavier NX là bước nâng cấp đáng kể so với Nano, với hiệu năng vượt trội được đóng gói trong module nhỏ gọn. Đây là lựa chọn phổ biến cho các ứng dụng thương mại cần hiệu năng tốt với ngân sách hợp lý.

- **CPU:** 6-core NVIDIA Carmel ARM v8.2 64-bit
- **GPU:** 384-core Volta với 48 Tensor Cores
- **RAM:** 8GB hoặc 16GB LPDDR4x
- **DLA:** 2x Deep Learning Accelerator
- **TDP:** 10W / 15W / 20W (tùy chỉnh)
- **Khả năng AI:** Lên đến 21 TOPS

Jetson Orin Nano và Orin NX

Dòng Orin là thế hệ mới nhất (2023), dựa trên kiến trúc Ampere của NVIDIA. Đây là dòng mạnh nhất trong phân khúc edge, với hiệu năng AI tăng gấp nhiều lần so với Xavier.

- **Orin Nano:** 20 TOPS, phù hợp thiết bị IoT thông minh.
- **Orin NX 8GB:** 70 TOPS
- **Orin NX 16GB:** 100 TOPS
- **Orin AGX:** Lên đến 275 TOPS, dành cho robotics và xe tự hành

Dòng Orin hỗ trợ đầy đủ các tính năng của CUDA 11.x, TensorRT 8.x, và tích hợp NPU (Neural Processing Unit) chuyên dụng giúp tăng tốc độ inference vượt trội.

2.1.3 Kiến trúc phần cứng Jetson

Kiến trúc của Jetson bao gồm các thành phần chính sau:

CPU ARM

CPU trong Jetson thuộc dòng ARM Cortex-A (các dòng cũ như Nano, TX2) hoặc ARM Carmel (Xavier) hoặc Cortex-A78AE (Orin). CPU xử lý các tác vụ điều phối, tiền xử lý dữ liệu, và các tác vụ không yêu cầu song song hoá cao.

GPU CUDA

Đây là thành phần quan trọng nhất cho AI. GPU trong Jetson thuộc các kiến trúc:

- **Maxwell** (Nano): Kiến trúc đời đầu, hỗ trợ CUDA nhưng chưa có Tensor Cores.
- **Volta** (Xavier): Lần đầu tích hợp Tensor Cores trong dòng Jetson.
- **Ampere** (Orin): Thế hệ mới nhất, hiệu năng vượt trội với nhiều Tensor Cores hơn và hỗ trợ sparsity.

Tensor Cores

Tensor Cores là đơn vị tính toán chuyên biệt cho phép thực hiện phép nhân ma trận (matrix multiplication) với độ chính xác hỗn hợp (FP16/BF16/INT8) ở tốc độ cực cao. Đây là nền tảng để tăng tốc các lớp tích chập (Convolutional Layer) và lớp fully-connected trong mạng neural.

DLA — Deep Learning Accelerator

DLA là bộ xử lý chuyên dụng cho inference, tiêu thụ ít điện năng hơn GPU nhưng vẫn đạt hiệu năng cao cho các model được hỗ trợ. Với TensorRT, người dùng có thể chỉ định một phần của model chạy trên DLA và phần còn lại chạy trên GPU.

Bộ nhớ thống nhất (Unified Memory)

Một điểm đặc trưng của Jetson là CPU và GPU chia sẻ cùng một không gian bộ nhớ vật lý (Unified Memory Architecture). Điều này loại bỏ chi phí copy dữ liệu giữa CPU và GPU, giúp giảm latency và tối ưu băng thông bộ nhớ.

2.1.4 Hệ sinh thái phần mềm của Jetson

NVIDIA cung cấp JetPack SDK — bộ công cụ phần mềm hoàn chỉnh bao gồm:

- **L4T (Linux for Tegra):** Hệ điều hành Ubuntu tùy chỉnh cho Jetson.
- **CUDA Toolkit:** Thư viện lập trình GPU.
- **cuDNN:** Thư viện tối ưu các phép tính deep learning trên CUDA.
- **TensorRT:** Framework tối ưu hoá inference (sẽ được trình bày chi tiết ở chương sau).
- **VisionWorks / OpenCV:** Xử lý ảnh và video.
- **DeepStream:** SDK xử lý video AI theo pipeline, hỗ trợ nhiều luồng video đồng thời.
- **Triton Inference Server:** Phục vụ model theo chuẩn REST/gRPC.

2.1.5 Ưu điểm của Jetson

- Hỗ trợ CUDA và toàn bộ hệ sinh thái NVIDIA: TensorRT, cuDNN, NCCL.
- Tích hợp phần cứng chuyên dụng (Tensor Cores, DLA) giúp tối ưu inference.
- Hiệu năng AI hàng đầu trong phân khúc Edge.
- JetPack SDK cung cấp đầy đủ công cụ phát triển.
- Hỗ trợ đa dạng giao tiếp: PCIe, USB 3.x, CSI camera, HDMI, GbE.
- Cộng đồng lập trình viên và tài liệu phong phú từ NVIDIA.

2.1.6 Nhược điểm của Jetson

- Giá thành cao hơn đáng kể so với Raspberry Pi (Jetson Nano khoảng \$100–\$150, Orin NX có thể lên đến \$400–\$700).
- Tiêu thụ điện nhiều hơn, cần nguồn điện ổn định.
- Cần carrier board phù hợp, tăng chi phí tổng thể.
- Cấu hình và debug phức tạp hơn với người mới bắt đầu.

2.2 Raspberry Pi

2.2.1 Tổng quan

Raspberry Pi là máy tính một bo mạch (Single Board Computer - SBC) được phát triển bởi Raspberry Pi Foundation (Anh), ra mắt lần đầu năm 2012 với mục đích giáo dục lập trình. Qua nhiều thế hệ, Raspberry Pi đã trở thành nền tảng phổ biến nhất trong cộng đồng maker, IoT, và các ứng dụng Edge Computing nhẹ.

Không giống Jetson được thiết kế đặc biệt cho AI, Raspberry Pi là máy tính đa dụng với khả năng chạy một hệ điều hành Linux đầy đủ. Tuy không có GPU mạnh để chạy deep learning, Raspberry Pi có thể được mở rộng với các thiết bị gia tốc AI ngoài như Coral USB Accelerator hay Intel Neural Compute Stick.

2.2.2 Các dòng Raspberry Pi phổ biến

Raspberry Pi 4 Model B

Đây là phiên bản phổ biến nhất hiện tại, với nhiều cải tiến đáng kể so với thế hệ trước:

- **CPU:** Broadcom BCM2711, Quad-core Cortex-A72 @ 1.5 GHz (có thể OC lên 2.0 GHz)
- **GPU:** VideoCore VI (chủ yếu dùng cho video decode, không hỗ trợ CUDA)
- **RAM:** 1GB / 2GB / 4GB / 8GB LPDDR4-3200
- **Kết nối:** Gigabit Ethernet, WiFi 5 (802.11ac), Bluetooth 5.0, 2x USB 3.0, 2x USB 2.0, 2x micro-HDMI
- **TDP:** 5–7W điển hình
- **Giá:** ~\$35 (1GB) đến \$75 (8GB)

Raspberry Pi 5

Ra mắt cuối năm 2023, Raspberry Pi 5 mang lại cải thiện hiệu năng lớn:

- **CPU:** Broadcom BCM2712, Quad-core Cortex-A76 @ 2.4 GHz
- **GPU:** VideoCore VII
- **RAM:** 4GB / 8GB LPDDR4X-4267
- **Tính năng mới:** PCIe 2.0 x1 (cho phép gắn SSD NVMe hoặc Coral M.2 Accelerator)
- **Giá:** \$60 (4GB) / \$80 (8GB)

Raspberry Pi Zero 2 W

Phiên bản nhỏ gọn, giá rẻ cho các ứng dụng IoT siêu nhỏ:

- **CPU:** Quad-core Cortex-A53 @ 1GHz
- **RAM:** 512MB LPDDR2
- **Kích thước:** 65mm × 30mm
- **Giá:** \$15

Raspberry Pi CM4 (Compute Module 4)

Phiên bản module dành cho tích hợp sản xuất công nghiệp, không có cổng ra mà phải dùng với carrier board tùy chỉnh. Phổ biến trong các sản phẩm thương mại.

2.2.3 Đặc điểm kỹ thuật

CPU ARM

CPU trong Raspberry Pi thuộc dòng ARM Cortex-A, chạy Instruction Set ARMv8 (64-bit). Đây là CPU đa mục đích, không có tối ưu đặc biệt cho deep learning. Tuy nhiên, từ kiến trúc Cortex-A72 (RPi 4) trở đi đã hỗ trợ NEON SIMD instructions giúp tăng tốc một số phép tính vector.

GPU VideoCore

GPU VideoCore trong Raspberry Pi được thiết kế chủ yếu cho:

- Decode video phần cứng (H.264, H.265 trên RPi 4/5)
- Hiển thị đồ họa 2D/3D đơn giản (OpenGL ES)

GPU này **không hỗ trợ CUDA** và không có Tensor Cores, nên không thể sử dụng TensorRT hay các thư viện AI NVIDIA.

GPIO và giao tiếp

Raspberry Pi có 40-pin GPIO (General Purpose Input/Output), hỗ trợ:

- I2C, SPI, UART (kết nối cảm biến, module)
- PWM (điều khiển động cơ, servo)
- Camera Interface (CSI) cho camera module chính thức

2.2.4 Các phương án mở rộng AI cho Raspberry Pi

Do hạn chế phần cứng, người dùng thường bổ sung các thiết bị gia tốc AI ngoài:

Google Coral USB Accelerator / Edge TPU

- Kết nối qua USB 3.0 hoặc M.2 (PCIe).
- Thực hiện inference trên mô hình TensorFlow Lite đã được compile cho Edge TPU.
- Hiệu năng: ~ 4 TOPS, tiêu thụ ~ 2 W.
- Phù hợp cho các model nhỏ (MobileNet, EfficientNet-Lite).

Intel Neural Compute Stick 2 (NCS2)

- Kết nối qua USB 3.0.
- Chạy mô hình được tối ưu bởi Intel OpenVINO.
- Hiệu năng: ~ 1 TOPS, hỗ trợ FP16.

Hailo-8 AI Accelerator

- Kết nối qua PCIe (M.2 hoặc HAT cho RPi 5).
- Hiệu năng: 26 TOPS — mạnh nhất trong các lựa chọn cho Raspberry Pi.
- Giá cao hơn, khoảng \$70–\$100.

2.2.5 Hệ điều hành và phần mềm

- **Raspberry Pi OS:** Hệ điều hành chính thức, dựa trên Debian Linux. Có phiên bản 32-bit và 64-bit.
- **Ubuntu Server / Desktop:** Hỗ trợ đầy đủ Ubuntu 22.04 LTS.
- **TensorFlow Lite:** Phiên bản nhẹ của TensorFlow, tối ưu cho thiết bị ARM.
- **ONNX Runtime:** Hỗ trợ chạy model ONNX trực tiếp trên CPU ARM.
- **OpenCV:** Xử lý ảnh và video.
- **PyTorch:** Có thể cài đặt (phiên bản ARM), nhưng chậm khi inference trên CPU.

2.2.6 Ưu điểm của Raspberry Pi

- Giá rẻ, dễ tiếp cận (từ \$15 đến \$80).
- Cộng đồng người dùng khổng lồ, tài liệu phong phú.
- Hỗ trợ đa dạng hệ điều hành Linux.
- Dễ tích hợp cảm biến và thiết bị ngoại vi qua GPIO.
- Tiêu thụ điện thấp, phù hợp cho hệ thống chạy liên tục.
- Kích thước nhỏ gọn, dễ tích hợp vào sản phẩm.

2.2.7 Nhược điểm của Raspberry Pi

- Không có CUDA, không thể chạy TensorRT hay cuDNN natively.
- Hiệu năng inference yếu so với Jetson khi không có accelerator ngoài.
- Bộ nhớ bằng thông thấp hơn.
- Không phù hợp cho real-time AI nặng (object detection nhiều luồng video, LLM, ...).
- Nếu cần thêm Coral/Hailo, chi phí tổng thể tăng đáng kể.

2.3 So sánh Jetson và Raspberry Pi

Tiêu chí	Jetson Orin NX 16GB	Raspberry Pi 5 (8GB)
CPU	Cortex-A78AE 8-core	Cortex-A76 4-core
GPU	Ampere 1024-core + TC	VideoCore VII
CUDA	Có	Không
TensorRT	Có	Không
AI Performance	100 TOPS	~0 (không có NPU)
RAM	16GB LPDDR5	8GB LPDDR4X
Giá (approx.)	\$400–\$600	\$80
TDP	10–25W	5–12W
Ứng dụng AI	Nặng, real-time	Nhẹ (cần thêm accelerator)

Chương 3

Pipeline triển khai AI

3.1 Tổng quan quy trình

Một quy trình triển khai model AI hoàn chỉnh từ nghiên cứu đến sản xuất thường bao gồm các bước sau:

1. **Chuẩn bị dữ liệu:** Thu thập, gán nhãn, augment dữ liệu huấn luyện.
2. **Thiết kế kiến trúc model:** Lựa chọn backbone, head phù hợp bài toán.
3. **Huấn luyện (Training):** Dùng framework như PyTorch hoặc TensorFlow trên GPU server.
4. **Đánh giá và tinh chỉnh:** Kiểm tra accuracy, recall, mAP, ... trên tập validation.
5. **Export sang ONNX:** Chuyển model sang định dạng trung gian độc lập framework.
6. **Tối ưu hoá với TensorRT:** Build engine tối ưu cho GPU mục tiêu (nếu dùng Jetson).
7. **Quantization** (tùy chọn): Giảm độ chính xác ($FP32 \rightarrow FP16 \rightarrow INT8$) để tăng tốc.
8. **Kiểm thử trên thiết bị:** Đo latency, throughput, độ chính xác thực tế.
9. **Deploy và monitoring:** Tích hợp vào hệ thống, theo dõi hiệu năng runtime.

3.2 Chi tiết từng bước

3.2.1 Chuẩn bị dữ liệu

Chất lượng dữ liệu quyết định phần lớn hiệu năng model. Các bước bao gồm:

- Thu thập ảnh/video đại diện cho điều kiện thực tế (lighting, angle, occlusion).
- Gán nhãn bằng công cụ như Roboflow, LabelImg, CVAT.
- Data augmentation: flip, rotate, crop, mosaic, color jitter để tăng tính tổng quát.
- Chia tập train/val/test với tỉ lệ hợp lý (ví dụ: 80/10/10).

3.2.2 Huấn luyện trên môi trường phát triển

Quá trình train thường thực hiện trên máy chủ GPU mạnh (NVIDIA A100, RTX 4090, ...) hoặc cloud (AWS, GCP). Framework phổ biến nhất hiện nay là **PyTorch**. Kết quả là một file model trọng số định dạng **.pt**.

```
import torch
import torch.nn as nn
import torch.optim as optim

# Khởi tạo model, loss, optimizer
model = MyModel().cuda()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)

# Training loop
for epoch in range(num_epochs):
    for batch_images, batch_labels in train_loader:
        images = batch_images.cuda()
        labels = batch_labels.cuda()

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

# Lưu model
torch.save(model.state_dict(), "weights/model_final.pt")
```

Listing 3.1: Ví dụ training loop đơn giản với PyTorch

3.2.3 Export sang ONNX

Sau khi training, model được export sang ONNX để chuẩn bị triển khai:

```
import torch
import torch.onnx

# Load model đã train
model = MyModel()
model.load_state_dict(torch.load("weights/model_final.pt"))
model.eval()

# Tạo dummy input (phải khớp với input shape của model)
dummy_input = torch.randn(1, 3, 640, 640) # batch=1, C=3, H=640, W=640

# Export
torch.onnx.export(
```

```

model,
dummy_input,
"model.onnx",
opset_version=17,
input_names=["images"],
output_names=["output"],
dynamic_axes={
    "images": {0: "batch_size"},
    "output": {0: "batch_size"}
},
export_params=True
)
print("Export ONNX thanh cong!")

```

Listing 3.2: Export model PyTorch sang ONNX

3.2.4 Tối ưu hoá với TensorRT

Trên Jetson (hoặc server NVIDIA), ONNX model được compile thành TensorRT engine:

```

# Dung trtexec (co san trong TensorRT installation)
trtexec \
    --onnx=model.onnx \
    --saveEngine=model_fp16.engine \
    --fp16 \
    --minShapes=images:1x3x640x640 \
    --optShapes=images:4x3x640x640 \
    --maxShapes=images:8x3x640x640 \
    --workspace=4096 \
    --verbose

# Neu muon INT8 quantization (can calibration data)
trtexec \
    --onnx=model.onnx \
    --saveEngine=model_int8.engine \
    --int8 \
    --calib=calibration_data/ \
    --workspace=4096

```

Listing 3.3: Build TensorRT engine từ ONNX

3.2.5 Inference trên thiết bị

Sau khi có engine file, inference được thực hiện bằng TensorRT Python/C++ API:

```

import tensorrt as trt
import pycuda.driver as cuda
import pycuda.autoinit
import numpy as np

# Load engine
TRT_LOGGER = trt.Logger(trt.Logger.WARNING)

```

```

with open("model_fp16.engine", "rb") as f:
    engine_data = f.read()

runtime = trt.Runtime(TRT_LOGGER)
engine = runtime.deserialize_cuda_engine(engine_data)
context = engine.create_execution_context()

# Allocate buffers
input_shape = (1, 3, 640, 640)
input_data = np.random.rand(*input_shape).astype(np.float32)

# Tao CUDA buffers va chay inference
# ... (allocate device memory, transfer, execute, transfer back)

```

Listing 3.4: Inference bằng TensorRT Python API

Chương 4

Các định dạng mô hình

4.1 PyTorch Model (.pt / .pth)

4.1.1 Khái niệm và cấu trúc

File **.pt** (hoặc **.pth**) là định dạng lưu model gốc của PyTorch, sử dụng thư viện **pickle** của Python để serialize các đối tượng Python. Đây là định dạng được sử dụng xuyên suốt quá trình nghiên cứu và phát triển model.

PyTorch cung cấp hai cách lưu model chính:

Cách 1: Lưu State Dict (Khuyến nghị)

State dict là một Python dictionary ánh xạ tên layer với tensor trọng số tương ứng. Đây là cách lưu được khuyến nghị vì tính linh hoạt và khả năng tương thích.

```
# LƯU
torch.save(model.state_dict(), "model_weights.pt")

# LOAD - cần khôi tạo model trước
model = MyModel()
model.load_state_dict(torch.load("model_weights.pt"))
model.eval()
```

Listing 4.1: Lưu và load bằng State Dict

Cách 2: Lưu toàn bộ model

Cách này lưu cả kiến trúc model và trọng số trong cùng một file:

```
# LƯU toàn bộ model (bao gồm cả kiến trúc)
torch.save(model, "model_full.pt")

# LOAD - không cần định nghĩa class trước
# (nhưng cần import các class con được sử dụng trong model)
model = torch.load("model_full.pt")
model.eval()
```

Listing 4.2: Lưu và load toàn bộ model

TorchScript

TorchScript là cách lưu model đặc biệt, chuyển model PyTorch thành intermediate representation có thể chạy độc lập không cần Python runtime. Có hai cách tạo TorchScript:

```
# Cach 1: Tracing (ghi lai cac phep tinh khi chay voi input mau)
traced_model = torch.jit.trace(model, dummy_input)
traced_model.save("model_traced.pt")

# Cach 2: Scripting (phan tich truc tiep source code Python)
scripted_model = torch.jit.script(model)
scripted_model.save("model_scripted.pt")

# Load TorchScript model
loaded_model = torch.jit.load("model_traced.pt")
```

Listing 4.3: TorchScript - Tracing và Scripting

4.1.2 Cấu trúc nội bộ của file .pt

File .pt thực chất là một file ZIP (bạn có thể mở bằng phần mềm giải nén) chứa:

- data.pkl: Pickle data chứa cấu trúc đối tượng Python.
- data/0: Binary tensor data cho mỗi tensor trọng số.
- record_size: Thông tin kích thước.

4.1.3 Sử dụng model .pt với YOLOv8

Các framework phổ biến như Ultralytics YOLOv8 sử dụng .pt làm định dạng chính:

```
from ultralytics import YOLO

# Load pretrained model
model = YOLO("yolov8n.pt") # nano variant

# Train tren dataset tùy chỉnh
results = model.train(
    data="coco128.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    device=0 # GPU index
)

# Inference
results = model.predict("test_image.jpg", conf=0.5, iou=0.45)

# Export sang ONNX
model.export(format="onnx", imgsz=640, opset=17)

# Export sang TensorRT
```

```
model.export(format="engine", imgsz=640, half=True, device=0)
```

Listing 4.4: YOLOv8 - Train và Inference với .pt

4.1.4 Ưu điểm của .pt

- Dễ dàng save và load trong môi trường PyTorch.
- Hỗ trợ đầy đủ tất cả tính năng PyTorch: custom layers, dynamic shapes, control flow.
- Dễ debug và visualize (dùng TensorBoard, Weights & Biases).
- Tích hợp tốt với các training framework (PyTorch Lightning, Hugging Face, Ultralytics).
- Có thể tiếp tục train (resume) từ checkpoint.
- Lưu được cả optimizer state, epoch number, loss history nếu cần.

4.1.5 Nhược điểm của .pt

- Phụ thuộc chặt chẽ vào Python và PyTorch runtime.
- Tốc độ inference chậm hơn nhiều so với TensorRT engine.
- Không portable sang các ngôn ngữ/framework khác mà không qua bước chuyển đổi.
- File state dict không chứa kiến trúc model, phải có source code để load.
- Không tối ưu bộ nhớ GPU khi inference.

4.2 ONNX Model (.onnx)

4.2.1 Khái niệm và lịch sử

ONNX (Open Neural Network Exchange) là định dạng mở cho việc biểu diễn mô hình học máy, được phát triển bởi Microsoft và Facebook (Meta) vào năm 2017. Mục tiêu của ONNX là tạo ra một ngôn ngữ chung để các mô hình có thể dễ dàng di chuyển giữa các framework (PyTorch, TensorFlow, MXNet, Caffe2, ...) và các backend inference khác nhau.

Ngày nay, ONNX được hỗ trợ bởi hầu hết các framework deep learning lớn và là bước trung gian bắt buộc trong nhiều pipeline triển khai.

4.2.2 Cấu trúc của file ONNX

File ONNX sử dụng định dạng Protocol Buffers (protobuf) để serialize một computation graph. Cấu trúc bao gồm:

- **Graph:** Đồ thị tính toán gồm các nodes và edges.
- **Node:** Mỗi node tương ứng với một phép toán (op) như Conv, BatchNorm, ReLU, Reshape, ...

- **Initializer:** Tensor trọng số (weights, biases).
- **Input/Output:** Thông tin về input và output của model.
- **Opset:** Phiên bản tập hợp các operators được sử dụng (ví dụ: opset 17).

4.2.3 Kiểm tra và visualize ONNX model

```
import onnx
import onnxruntime as ort

# Kiểm tra tính hợp lệ của model
model = onnx.load("model.onnx")
onnx.checker.check_model(model)
print("Model ONNX hợp lệ!")

# In ra thông tin graph
print(onnx.helper.printable_graph(model.graph))

# Lay thông tin input/output
for input in model.graph.input:
    print(f"Input: {input.name}, shape: {[d.dim_value for d in
        input.type.tensor_type.shape.dim]}")
for output in model.graph.output:
    print(f"Output: {output.name}")
```

Listing 4.5: Kiểm tra và inspect ONNX model

```
import onnxruntime as ort
import numpy as np

# Tao session - tu dong chon provider tot nhat
providers = ['CudaExecutionProvider', 'CPUExecutionProvider']
session = ort.InferenceSession("model.onnx", providers=providers)

# In ra input info
for inp in session.get_inputs():
    print(f"Input name: {inp.name}, shape: {inp.shape}, dtype: {
        inp.type}")

# Chay inference
input_data = np.random.randn(1, 3, 640, 640).astype(np.float32)
outputs = session.run(None, {"images": input_data})

print(f"Output shape: {outputs[0].shape}")
```

Listing 4.6: Inference bằng ONNX Runtime

4.2.4 ONNX Simplifier

Đôi khi model ONNX được export ra có nhiều node dư thừa (như Cast, Gather, Unsqueeze không cần thiết). ONNX Simplifier giúp làm sạch graph:

```

pip install onnx-simplifier

python -m onnxsim model.onnx model_simplified.onnx

# Hoac su dung trong Python
from onnxsim import simplify
import onnx

model = onnx.load("model.onnx")
model_simplified, check = simplify(model)
assert check, "Simplification that"
onnx.save(model_simplified, "model_simplified.onnx")

```

Listing 4.7: Simplify ONNX model

4.2.5 ONNX Opset và tương thích

Mỗi operator trong ONNX có thể thay đổi định nghĩa qua các phiên bản opset. Một số lưu ý quan trọng:

- **Opset 11:** Hỗ trợ dynamic axes cơ bản.
- **Opset 13:** Cải thiện support cho nhiều operator.
- **Opset 17:** Phiên bản hiện tại, hỗ trợ nhiều tính năng mới nhất.
- TensorRT 8.x hỗ trợ tốt nhất với opset 11–17.
- Nên dùng opset cao nhất mà TensorRT backend hỗ trợ để tránh lỗi khi convert.

4.2.6 Các ONNX Execution Providers

ONNX Runtime hỗ trợ nhiều backends (gọi là Execution Providers):

- **CPUExecutionProvider:** Chạy trên CPU (mặc định).
- **CUDAExecutionProvider:** Chạy trên GPU NVIDIA.
- **TensorrtExecutionProvider:** Tự động tối ưu một phần bằng TensorRT.
- **CoreMLExecutionProvider:** Trên Apple Silicon / iOS.
- **OpenVINOExecutionProvider:** Trên Intel CPU/GPU/VPUs.

4.2.7 Ưu điểm của ONNX

- Cross-platform và cross-framework: chạy được trên nhiều thiết bị và OS.
- Là bước trung gian bắt buộc để convert sang TensorRT, OpenVINO, CoreML.
- ONNX Runtime cung cấp inference nhanh hơn PyTorch gốc trên CPU.
- Có thể visualize model graph bằng Netron (công cụ mở).
- Hỗ trợ dynamic batch size và dynamic input shapes.

4.2.8 Nhược điểm của ONNX

- Không phải tất cả custom operators đều được hỗ trợ (ví dụ: một số layer đặc biệt của transformer).
- Debug khó hơn PyTorch gốc khi xảy ra lỗi.
- Có thể xảy ra sai số nhỏ (numerical difference) so với kết quả PyTorch do cách implement operator.
- Tốc độ inference chậm hơn TensorRT engine đáng kể.
- Cần cẩn thận với opset version khi chuyển đổi giữa các tool.

4.3 TensorRT Engine (.engine / .trt)

4.3.1 Khái niệm và vai trò

TensorRT là framework tối ưu hoá inference của NVIDIA, được thiết kế để tối đa hoá thông lượng (throughput) và giảm thiểu độ trễ (latency) khi chạy model trên GPU NVIDIA. TensorRT thực hiện một loạt các tối ưu hoá graph và kernel để tạo ra một **engine** được compile đặc biệt cho GPU mục tiêu.

File **.engine** (còn gọi là **.trt**) là kết quả của quá trình này — một binary đã được tối ưu hoàn toàn cho phần cứng cụ thể, không thể dùng lại trên GPU khác.

4.3.2 Quy trình tối ưu hoá của TensorRT

TensorRT thực hiện nhiều loại tối ưu hoá:

1. Layer Fusion

TensorRT gộp nhiều layer liên tiếp lại thành một kernel duy nhất. Ví dụ, Conv + BatchNorm + ReLU có thể được gộp thành một phép tính duy nhất, giảm số lần truy cập bộ nhớ:

$$\text{Conv} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \implies \text{ConvBnReLU (1 kernel)}$$

2. Precision Calibration (Quantization)

TensorRT hỗ trợ nhiều mức độ chính xác:

- **FP32** (Float 32-bit): Độ chính xác đầy đủ, tốc độ cơ bản.
- **FP16** (Float 16-bit): Nhanh gấp $\sim 2x$ trên GPU có Tensor Cores, mất rất ít độ chính xác ($< 0.5\%$ mAP).
- **INT8** (Integer 8-bit): Nhanh nhất (gấp $\sim 4x$ so với FP32), cần calibration data để giảm thiểu mất mát độ chính xác.
- **INT4** (Experimental): Hỗ trợ trên các GPU mới nhất (Hopper, Ada).

Quá trình INT8 calibration:

```

import tensorrt as trt
import numpy as np

class MyCalibrator(trt.IInt8EntropyCalibrator2):
    def __init__(self, data_loader, cache_file="calibration.cache"):
        super().__init__()
        self.data_loader = iter(data_loader)
        self.cache_file = cache_file
        # Allocate GPU buffer
        self.device_input = cuda.mem_alloc(batch_size * 3 * 640 *
            640 * 4)

    def get_batch(self, names):
        try:
            batch = next(self.data_loader)
            cuda.memcpy_htod(self.device_input, batch.numpy())
            return [int(self.device_input)]
        except StopIteration:
            return None

    def read_calibration_cache(self):
        if os.path.exists(self.cache_file):
            with open(self.cache_file, "rb") as f:
                return f.read()

    def write_calibration_cache(self, cache):
        with open(self.cache_file, "wb") as f:
            f.write(cache)

```

Listing 4.8: INT8 Calibration bằng TensorRT Python API

3. Kernel Auto-tuning

TensorRT thử nghiệm nhiều cách implement khác nhau cho mỗi phép toán (ví dụ: các thuật toán convolution khác nhau như GEMM, Winograd, FFT) và chọn cách tối ưu nhất cho GPU cụ thể. Đây là lý do quá trình build engine có thể mất vài phút.

4. Dynamic Tensor Memory

TensorRT phân tích memory usage của toàn bộ graph và sắp xếp lại bộ nhớ để giảm peak memory consumption.

5. Graph Optimization

- Loại bỏ các node dư thừa (Constant Folding).
- Tối ưu layout tensor (ví dụ: NCHW vs NHWC tùy GPU).
- Horizontal layer fusion: gộp các layer song song có cùng input.

4.3.3 Dynamic Shapes trong TensorRT

TensorRT engine có thể được build với dynamic batch size và dynamic input shapes:

```
trtexec \  
  --onnx=model.onnx \  
  --saveEngine=model_dynamic.engine \  
  --fp16 \  
  --minShapes=images:1x3x320x320 \  
  --optShapes=images:4x3x640x640 \  
  --maxShapes=images:16x3x1280x1280  
  
# minShapes: kích thước nhỏ nhất có thể xử lý  
# optShapes: kích thước tối ưu (sẽ được compile cho kích thước này)  
# maxShapes: kích thước lớn nhất có thể xử lý
```

Listing 4.9: Build engine với dynamic shapes

4.3.4 Inference với TensorRT C++ API

```
#include "NvInfer.h"  
#include "NvInferRuntime.h"  
#include <fstream>  
#include <vector>  
  
// Logger  
class Logger : public nvinfer1::ILogger {  
    void log(Severity severity, const char* msg) noexcept override  
    {  
        if (severity <= Severity::kWARNING)  
            std::cout << msg << std::endl;  
    }  
} logger;  
  
int main() {  
    // Doc engine file  
    std::ifstream file("model_fp16.engine", std::ios::binary);  
    std::vector<char> engineData((std::istreambuf_iterator<char>(file)),  
                                std::istreambuf_iterator<char>());  
  
    // Tao runtime va deserialize engine  
    auto runtime = nvinfer1::createInferRuntime(logger);  
    auto engine = runtime->deserializeCudaEngine(engineData.data(),  
        engineData.size());  
    auto context = engine->createExecutionContext();  
  
    // Allocate GPU buffers  
    // ... (cuda malloc, memcpy, enqueueV3, cudaStreamSynchronize)
```

```
    return 0;
}
```

Listing 4.10: TensorRT Inference với C++

4.3.5 Kiểm tra hiệu năng với trtexec

```
# Benchmark engine đã build
trtexec \
  --loadEngine=model_fp16.engine \
  --batch=1 \
  --iterations=1000 \
  --warmUp=100 \
  --percentile=99

# Output sẽ hiển thị:
# - Mean latency (ms)
# - 99th percentile latency (ms)
# - Throughput (inferences/sec)
```

Listing 4.11: Benchmark engine bằng trtexec

4.3.6 Tương thích phiên bản

Một điểm quan trọng cần lưu ý: file .engine được compile bởi một phiên bản TensorRT và GPU cụ thể sẽ **không chạy được** trên GPU khác hoặc phiên bản TensorRT khác. Ví dụ:

- Engine build trên Jetson Orin NX (TensorRT 8.5) **không dùng được** trên Jetson Nano (TensorRT 7.x).
- Engine build trên RTX 3090 **không dùng được** trên Jetson Orin.

Vì vậy, cần **build engine trực tiếp trên thiết bị mục tiêu** hoặc dùng cross-compilation với đúng toolchain.

4.3.7 Ưu điểm của TensorRT Engine

- Tốc độ inference nhanh nhất có thể đạt được trên GPU NVIDIA.
- Tận dụng tối đa Tensor Cores, DLA và các tính năng phần cứng đặc biệt.
- Hỗ trợ FP16 và INT8 với mức độ chính xác tối thiểu.
- Giảm latency và tăng throughput đáng kể so với PyTorch/ONNX.
- Hỗ trợ dynamic batching và dynamic shapes.
- Tích hợp tốt với NVIDIA DeepStream cho xử lý video.

4.3.8 Nhược điểm của TensorRT Engine

- Không portable: phải build riêng cho từng GPU và phiên bản TensorRT.
- Quá trình build engine tốn thời gian (có thể vài chục phút với model lớn).
- Chỉ hoạt động trên phần cứng NVIDIA.
- Một số custom operators không được hỗ trợ, cần implement TensorRT plugin.
- Khó debug khi có sai số kết quả so với PyTorch gốc.
- INT8 quantization cần dữ liệu calibration đại diện, nếu không có thể giảm độ chính xác.

Chương 5

So sánh các định dạng mô hình

5.1 So sánh tổng quan

Tiêu chí	.pt	.onnx	.engine
Tốc độ inference	Thấp	Trung bình	Cao nhất
Portability	Thấp	Cao	Rất thấp
Phụ thuộc phần cứng	Không	Không	Có (GPU cụ thể)
Dễ debug	Cao	Trung bình	Thấp
Hỗ trợ training	Có	Không	Không
Hỗ trợ FP16/INT8	Không đầy đủ	Qua backend	Có đầy đủ
Cross-framework	Không	Có	Không
Thời gian build	Nhanh	Nhanh	Chậm
Sử dụng chính	Training	Chuyển đổi	Deploy NVIDIA

5.2 So sánh hiệu năng thực tế

Dưới đây là ví dụ kết quả benchmark điển hình chạy YOLOv8n trên Jetson Orin NX 16GB với input 640×640 :

Định dạng	Latency (ms)	FPS	Độ chính xác (mAP)
PyTorch (.pt) FP32	~45 ms	~22	37.3
ONNX Runtime FP32	~30 ms	~33	37.3
TensorRT FP32	~12 ms	~83	37.3
TensorRT FP16	~6 ms	~167	37.0 (-0.3)
TensorRT INT8	~4 ms	~250	36.5 (-0.8)

Lưu ý: Các con số trên là ước lượng minh họa, kết quả thực tế phụ thuộc nhiều vào thiết bị, phiên bản phần mềm và model cụ thể.

5.3 Khi nào nên dùng định dạng nào

5.3.1 Dùng .pt khi

- Đang trong giai đoạn nghiên cứu, thử nghiệm kiến trúc model.

- Cần tiếp tục train hoặc fine-tune model.
- Debug và visualize activation, gradient.
- Chạy inference trên máy tính phát triển có RAM GPU lớn.

5.3.2 Dùng .onnx khi

- Cần chạy model trên nhiều thiết bị và platform khác nhau.
- Chạy inference trên CPU (dùng ONNX Runtime CPU).
- Cần deploy lên các thiết bị không có NVIDIA GPU (Intel OpenVINO, Apple CoreML).
- Làm bước trung gian để convert sang TensorRT hoặc các format khác.
- Ứng dụng đòi hỏi portability cao.

5.3.3 Dùng .engine khi

- Deploy sản phẩm trên thiết bị NVIDIA (Jetson, server GPU).
- Yêu cầu real-time inference với latency thấp nhất có thể.
- Xử lý nhiều luồng video đồng thời (kết hợp với DeepStream).
- Ứng dụng production cần throughput cao.
- Tài nguyên GPU hạn chế, cần tối ưu bộ nhớ.

Chương 6

Ứng dụng thực tế

6.1 Ứng dụng của Jetson

6.1.1 Xe tự hành và Robot

Jetson AGX Orin là nền tảng được lựa chọn cho nhiều hệ thống xe tự hành cấp độ nghiên cứu và thương mại. Các tác vụ bao gồm:

- 3D object detection từ LiDAR và camera (fusion).
- Semantic segmentation đường đi.
- Lane detection và tracking.
- Pedestrian và obstacle detection real-time.
- SLAM (Simultaneous Localization and Mapping).

NVIDIA cũng cung cấp DRIVE platform (dựa trên Jetson) chuyên cho xe tự hành.

6.1.2 Hệ thống Camera AI thông minh

- **Face recognition:** Nhận diện khuôn mặt trong hệ thống kiểm soát vào ra.
- **Crowd analytics:** Đếm người, phân tích luồng người trong trung tâm thương mại.
- **PPE detection:** Phát hiện thiếu thiết bị bảo hộ trong công trường.
- **Defect detection:** Kiểm tra lỗi sản phẩm trên dây chuyền sản xuất.
- **License plate recognition:** Nhận dạng biển số xe tự động.

6.1.3 Smart City và Giám sát giao thông

- Đếm và phân loại phương tiện giao thông.
- Phát hiện vi phạm giao thông (vượt đèn đỏ, đi sai làn).
- Phân tích mật độ giao thông để tối ưu đèn điều tiết.
- Phát hiện sự cố giao thông tự động.

6.1.4 Y tế và Thiết bị y tế

- Phân tích ảnh X-ray, CT scan tại chỗ (không cần gửi lên cloud).
- Hỗ trợ phẫu thuật robot với AI guidance.
- Monitoring bệnh nhân real-time trong ICU.

6.1.5 Agriculture AI

- Phát hiện sâu bệnh trên cây trồng từ ảnh drone.
- Đếm và phân loại trái cây để hỗ trợ thu hoạch tự động.

6.2 Ứng dụng của Raspberry Pi

6.2.1 IoT và Home Automation

- Smart home hub: Điều khiển đèn, nhiệt độ, cửa thông qua MQTT/Home Assistant.
- Cảm biến môi trường: Thu thập và truyền dữ liệu nhiệt độ, độ ẩm, CO2.
- Tưới cây tự động dựa trên độ ẩm đất.
- Hệ thống chuông cửa thông minh với camera.

6.2.2 Edge AI nhẹ với Accelerator

- Phát hiện đối tượng đơn giản (với Coral TPU): phát hiện người, xe trong camera an ninh nhỏ.
- Nhận dạng giọng nói: Kích hoạt thiết bị bằng từ khoá (wake word detection).
- Phân loại ảnh đơn giản cho ứng dụng nông nghiệp quy mô nhỏ.

6.2.3 Giáo dục và Nghiên cứu

- Nền tảng học lập trình Python, Linux.
- Dự án STEM cho học sinh, sinh viên.
- Prototype nhanh các ý tưởng IoT.
- Máy chủ nhỏ, media center, retro gaming console.

6.2.4 Media và Entertainment

- Kiosk thông tin trong bảo tàng, triển lãm.
- Digital signage (biển quảng cáo kỹ thuật số).
- Hệ thống âm nhạc phòng chờ (Volumio, MusicBox).

Chương 7

Các lưu ý và thực hành tốt nhất

7.1 Lưu ý khi chọn phần cứng

- Đánh giá rõ yêu cầu về FPS và latency của ứng dụng trước khi chọn phần cứng.
- Tính tổng chi phí hệ thống (không chỉ giá board): nguồn điện, vỏ, cooling, storage, carrier board.
- Với ứng dụng cần xử lý nhiều luồng video (ví dụ: 4 camera $\times$ 30 FPS), chỉ Jetson mới đáp ứng được.
- Nếu không cần GPU mạnh, Raspberry Pi + Coral TPU có thể là lựa chọn kinh tế hơn cho bài toán đơn giản.

7.2 Lưu ý khi chuyển đổi định dạng model

- Luôn kiểm tra **numerical accuracy** sau mỗi bước chuyển đổi bằng cách so sánh output của model với cùng input.
- Dùng **ONNX Simplifier** trước khi convert sang TensorRT để loại bỏ node dư.
- Với model có **custom operators** (như các biến thể attention mới), cần implement TensorRT plugin tương ứng.
- **Không** dùng lại engine file đã build trên GPU A cho GPU B — phải build lại.
- Khi dùng INT8, bộ dữ liệu calibration phải **đại diện** cho phân phối dữ liệu thực tế.

7.3 Tối ưu hoá thêm sau TensorRT

- **CUDA Streams**: Chạy preprocess và inference song song với nhau.
- **CUDA Graphs**: Capture toàn bộ sequence CUDA calls thành graph để replay nhanh hơn.
- **Double buffering**: Dùng 2 buffer để overlap data transfer và compute.

- **Model pruning:** Cắt tỉa các weight không quan trọng trước khi quantize.
- **Knowledge Distillation:** Train model nhỏ hơn (student) từ model lớn (teacher) để giữ độ chính xác.

Chương 8

Kết luận

Tài liệu này đã trình bày chi tiết về hai nền tảng phần cứng Edge AI phổ biến và ba định dạng model quan trọng trong pipeline triển khai AI.

Về phần cứng:

- **Jetson** là lựa chọn hàng đầu khi cần hiệu năng AI cao, hỗ trợ CUDA và TensorRT, phù hợp cho các ứng dụng real-time đòi hỏi xử lý ảnh/video phức tạp.
- **Raspberry Pi** phù hợp cho các ứng dụng IoT nhẹ, prototype nhanh, và giáo dục, với chi phí thấp và cộng đồng hỗ trợ lớn. Có thể nâng cao khả năng AI bằng cách bổ sung accelerator ngoài.

Về định dạng model:

- **.pt** là định dạng dùng trong nghiên cứu và phát triển, linh hoạt nhưng không tối ưu cho deployment.
- **.onnx** là cầu nối giữa các framework và backend, đảm bảo portability và là bước không thể thiếu trong pipeline.
- **.engine** là đích đến cuối cùng khi deploy trên thiết bị NVIDIA, mang lại hiệu năng tối đa nhưng bị ràng buộc với phần cứng cụ thể.

Pipeline tối ưu cho deployment trên Jetson:

Train (.pt) $\xrightarrow{\text{export}}$ ONNX $\xrightarrow{\text{simplify}}$ ONNX Simplified $\xrightarrow{\text{trtexec}}$ TensorRT Engine

Việc nắm vững toàn bộ pipeline từ training đến deployment, hiểu rõ đặc điểm từng phần cứng và định dạng model, là nền tảng để xây dựng các hệ thống AI Edge hiệu quả, đáp ứng yêu cầu thực tế.